filter pipeline. These are obviously highly optimized, but if you're writing an application where you need to squeeze out every little bit of throughput, minimal APIs will likely help get you there more easily. For most applications, the overhead of the MVC framework will be negligible when compared with any database or network access in your app, so this is worth worrying about only for performance-sensitive apps.

Question 3 focuses on filtering. You learned about filtering with minimal APIs in chapter 5: filters allow you to attach a processing pipeline to your minimal API endpoints and can be used to do things like automatic validation. Web API controllers (as well as MVC controllers and Razor Pages) also have a filter pipeline, but it's *much* more complex than the simple pipeline used by minimal APIs, as you'll see in chapters 21 and 22.

In most cases the filtering provided by minimal APIs will be perfectly adequate for your needs. The main cases where minimal API filtering will fall down will be when you already have an application that uses web API controllers and want to reuse some complex filters. In these cases, there may be no way to translate your existing web API filters to minimal API filters. If the filtering is important, then you may need to stick with web API controllers.

This leads to question 4: are you building a new application or working on an existing application? If this is a new application, I would be strongly in favor of using minimal APIs. Minimal APIs are conceptually simpler than web API controllers, are faster because of this, and are receiving a lot of improvements from the ASP.NET Core team. If there's no other compelling reason to choose web API controllers in your new project, I suggest defaulting to minimal APIs.

On the other hand, if you have an existing web API controller application, I would be strongly inclined to stick with web API controllers. While it's